Общество с ограниченной ответственностью
«Центр повышения квалификации и профессиональной подготовки «Основание»

ВЫПУСКНАЯ АТТЕСТАЦИОННАЯ (ПРОЕКТНАЯ) РАБОТА

«Разработка приложения для отображения курсов криптовалют»

по программе профессиональной переподготовки
«Азбука цифры. Программирование на языке Python от базового уровня до продвинутого»

Выполнил:	_____	11.11.2026	Брагин М.А.
	подпись	***дата***	***инициалы, фамилия***

Москва 2026

1. ВВЕДЕНИЕ

Приложение для отображения курсов криптовалют» разрабатывается для пользователей, которым ежедневно необходима актуальная информация о курсах популярных криптовалют в исследовательских целях. Приложение позволяет в любой момент времени получить актуальные курсы валют, используя данные открытого API. Приложение имеет интерфейс на русском языке и не предназначено для распространения за пределами Российской Федерации. К работе с приложением допускаются пользователи не младше 14 лет.

2. ТЕХНИЧЕСКОЕ ЗАДАНИЕ

2.1. Общие сведения

2.1.1. Полное наименование системы – программное обеспечение «Курсы популярных криптовалют»

2.1.2. Организация – разработчик, и организация – заказчик

Разработчик: Брагин Михаил Александрович
Заказчик: ООО «Основание»

2.2. Назначение и цели создания ПО

2.2.1. Цели создания системы

Приложение разрабатывается для оперативного получения пользователями актуальной информации о курсах популярных на сегодняшний день криптовалют в исследовательских целях

2.2.2. Перечень работ по созданию системы

Данным техническим заданием требуется создать ПО и произвести следующие работы:

- Разработать функционал отображения пользовательского интерфейса;
- Разработать функционал обращения к сайту с открытым API;
- Разработать функционал вывода полученных данных о курсах валют в интерфейс пользователя;
- Обеспечить обработку исключений для повышения стабильности работы приложения.

2.2.3. Сведения о существующей технологии

В настоящее время для получения информации о курсах валют пользователям необходимо самостоятельно отыскать подходящий электронный ресурс в сети Интернет, найти соответствующий раздел с курсами валют и получить информацию, что не всегда удобно. Разрабатываемое ПО поможет пользователям узнавать курсы валют без поиска ресурса в Сети, а получать необходимые котировки сразу в интерфейсе приложения.

2.2.4. Характеристики объекта информатизации

Объект автоматизации

Объектом автоматизации является работа программного обеспечения «Курсы валют».

2.3. Требования к ПО

2.3.1. Описание типовых процессов

- автоматизируется процесс получения информации о курсах популярных криптовалют к доллару США из сети Интернет в реальном времени;

2.3.2. Требования к функциям, выполняемым ПО

- выбор базовой валюты из выпадающего списка;
- вывод информации о курсе обмена валют;
- изменение кода базовой валюты на новый (выбор валюты из списка)
- обновление курсов: автоматическое (через заданным промежуток времени) или ручное

2.3.3. Общие технические требования к ПО

- программа должна работать непрерывно до прекращения ее работы пользователем;

2.4. Требования к документации

В момент сдачи проекта заказчику предоставляется следующий набор документов:

- Руководство пользователя в электронном виде;
- Справочник с названиями популярных валют.

2.5. Порядок предоставления дистрибутива

По окончании разработки Исполнитель должен предоставить Заказчику дистрибутив системы в составе:

- архив с исходными кодами всех программных модулей и разделов;
- файл проекта в формате *.py
- дистрибутив последней версии Python
- пароль от архива

2.6. Защита исходного кода

Требуется защитить исходный код. Архив с исходными кодами должен быть заархивирован с паролем длиной не менее 10 символов.

2.7. Требования к техническому обеспечению

Требования к ПК:

- Компьютер с процессором Процессор Intel® Core™ i3 2 ГГц
- ОЗУ 1024 Мб (рекомендуется от 2048 Мб)
- программа работает под управлением ОС Windows;
- для работы ПО на компьютере должен быть установлен Python версии не ниже 3.11

2.8. Глоссарий и перечень сокращений

ПО – программное обеспечение
ПК – персональный компьютер
ОЗУ – оперативное запоминающее устройство (оперативная память ПК)

3. РЕЗУЛЬТАТЫ ИССЛЕДОВАНИЯ ОТКРЫТЫХ API, ПРЕДОСТАВЛЯЮЩИХ КУРСЫ ВАЛЮТ

3.1. Поиск открытых API

Для поиска открытых API использовалась поисковая система Yandex (ya.ru). По запросу «где можно узнать курсы криптовалют официально» было найдено более 10 сайтов с открытыми API. Ниже приведены ссылки на первые 5 сайтов со списками открытых API :

- CoinMarketCap — показывает цены, графики и рыночную капитализацию. Есть русскоязычная версия.
- CoinGecko — позволяет отслеживать более 10 000 монет в разных валютах (например, BTC/RUB, ETH/RUB).
- Blockchain.com — в разделе Explorer есть актуальные цены и данные о сделках.
- Coinbase — в разделе Explore тоже есть актуальные цены, графики и рыночная капитализация.
- Binance — на странице Markets/Overview можно увидеть курсы криптовалют, которые торгуются на бирже.

coinmarketcap.com
coingecko.com
blockchain.com
coinbase.com
binance.com

3.2. Поиск API с курсами валют

При анализе найденных сайтов были найдены следующие API, позволяющие получить курсы обмена валюты и не требующие ключа apiKey:

CoinGecko - url = "https://api.coingecko.com/api/v3/simple/price"

- Бесплатный Demo API: 10 000 запросов в месяц.
- До 100 запросов в минуту.
- Цена, капитализация, изменение за 24 часа.
- Для Demo требуется бесплатная регистрация.
- Сайт и документация · API Docs.

Binance - url = "https://data-api.binance.vision/api/v3/ticker/price"
- url = "https://api.binance.com/api/v3/ticker/price"

- Без API-ключа
- Публичные рыночные данные.
- Текущая цена и изменение за 24 часа.
- Не нужна регистрация.
- Данные конкретной биржи, а не средняя цена всего рынка.
- Документация.

CoinMarketCap
- url = "https://pro-api.coinmarketcap.com/public-api/v3/cryptocurrency/listings/latest"

- Публичные endpoints без ключа.
- Бесплатный Basic API с регистрацией.
- Цены и рыночная статистика.

3.3. Выбор конкретного API для использования в проекте

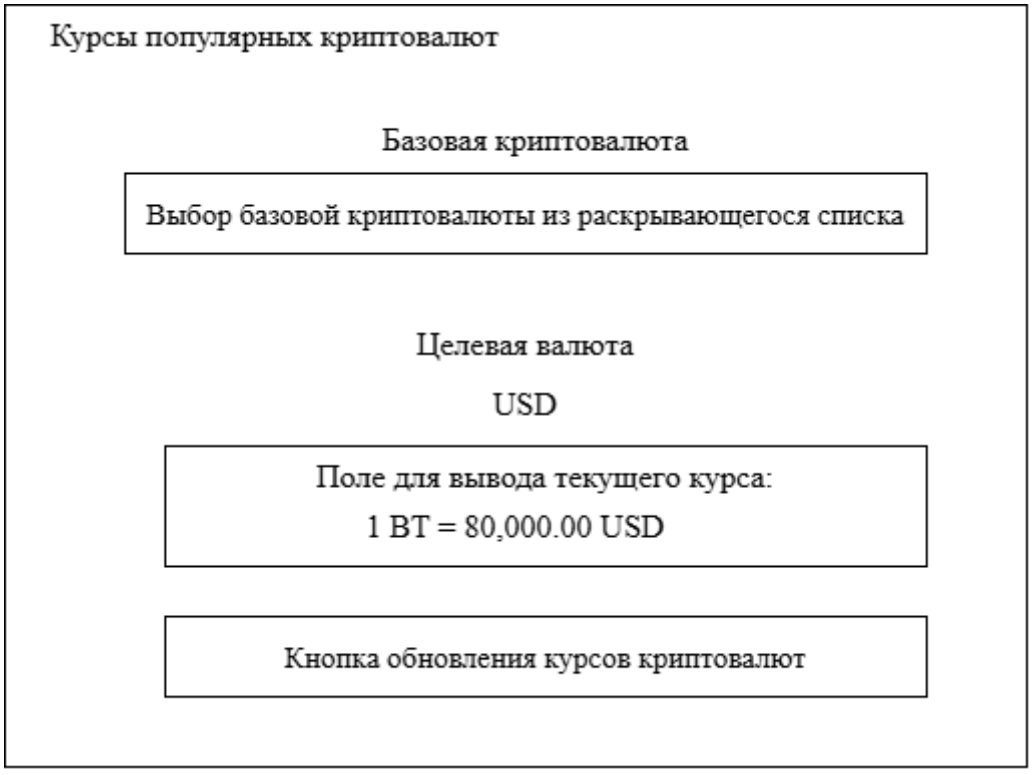
Был проведен анализ отзывов в интернет по всем исследуемым сервисам, в результате для использования в проекте был выбран сервис https://api.coingecko.com/api/v3/simple/price по следующим причинам:

- Стабильный и доступный ресурс, и не закрытый для ip адресов из России, можно задавать параметры для выбора конкретного списка валют с определенными требованиями (изменение курса за 24 часа и т.д.)
- Чтобы получить данные по курсам с сайта Binance, нужно сделать последовательно 10 запросов (по количеству валют), что может временно заблокировать сервис.
- Json-ответ от сайта CoinMarketCap содержит более 4500 строк информации, что затрудняет обработку

4. ТРЕБОВАНИЯ К ГРАФИЧЕСКОМУ ИНТЕРФЕЙСУ ПОЛЬЗОВАТЕЛЯ

4.1. Требования к интерфейсу

Базовый интерфейс ПО должен соответствовать приведенному предварительному эскизу:



Разработчик может вносить дополнения в данный эскиз по согласованию с заказчиком

4.2. Элементы интерфейса должны располагаться сверху вниз в следующем порядке:

- заголовок окна – «Курсы популярных криптовалют»;
- надпись – «Базовая криптовалюта»;
- выпадающий список для выбора базовой валюты;
- надпись – «Целевая валюта»;
- поле для вывода текущего курса выбранной валюты
- кнопка с надписью — «Обновить курсы криптовалют»

5. ТРЕБОВАНИЯ К ТЕСТИРОВАНИЮ

5.1. ПО должно быть протестировано перед отправкой заказчику. Тестирование проводится в соответствии с “Программой и методикой испытаний”.

6. ЛИСТИНГ

```
from tkinter import *
from tkinter import ttk
from tkinter import messagebox as mb
import requests
from datetime import datetime
import copy
import random
from time import sleep
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
from matplotlib.figure import Figure


def target_field_update(event=None):
    """Обновляет основной курс выбранной из списка криптовалюты"""
    if crypto_json:
        item = crypto_combo.get()
        base = crypto[item]
        try:
            rate = crypto_json[base]['usd']
            target_field.config(text=f'{item} = {rate:>10,.2f} USD')
        except KeyError:
            target_field.config(text=f'{'___'} = {'_' * 7}.00 USD')
            change24_value.config(text='-', fg='black')
            last_change_value.config(text='-', fg='black')
            mb.showerror('Ошибка', f'Валюта {item} отсутствует в ответе API')
    else:
        target_field.config(text='Ошибка при загрузке курсов', font=('Verdana', 11))

    change_24_update()
    change_last_update()
    root.after(500, diagram)
```

```
def change_24_update():
    """Показывает изменение курса за последние 24 часа в %"""
    if crypto_json:
        item = crypto_combo.get()
        base = crypto[item]
        try:
            value = crypto_json[base]['usd_24h_change']
            if value > 0:
                change24_value.config(text=f'↑ {value:+.2f} %', fg='green')
            elif value < 0:
                change24_value.config(text=f'↓ {value:.2f} %', fg='red')
            else:
                change24_value.config(text=f'= {value:.2f} %', fg='black')
        except KeyError:
            change24_value.config(text='-', fg='black')
    else:
        change24_value.config(text='-', fg='black')


def change_last_update():
    """Показывает изменение курса относительно предыдущего обновления в %"""
    if crypto_json and previous_crypto_json:
        item = crypto_combo.get()
        base = crypto[item]
        try:
            rate_prev = previous_crypto_json[base]['usd']
            rate = crypto_json[base]['usd']
            change = (rate - rate_prev) / rate_prev * 100
            if change > 0:
                last_change_value.config(text=f'↑ {change:+.2f} %', fg='green')
            elif change < 0:
                last_change_value.config(text=f'↓ {change:.2f} %', fg='red')
            else:
                last_change_value.config(text=f'= {change:.2f} %', fg='black')
        except KeyError:
            last_change_value.config(text='-', fg='black')
    else:
        last_change_value.config(text='-', fg='black')


def add_to_chart():
    """Создание словаря изменений котировок для графика и списка макс/мин значений для
значений оси Y"""
    global values_yaxis, max_value_yaxis, min_value_yaxis

    if crypto_json:
        for base_cur in crypto.values():
            if base_cur not in crypto_json:
                continue
            else:
                value = crypto_json[base_cur]['usd_24h_change']
                chart.setdefault(base_cur, {'Изменение за 24 часа': [], 'Изменение с предыдущего
```

```
запроса': [0.00]})

            values_yaxis.append(int(value))  # добавление значения в список для мин/макс оси
Y

            if len(chart[base_cur]['Изменение за 24 часа']) < 10:
                chart[base_cur]['Изменение за 24 часа'].append(round(value, 2))
            else:
                chart[base_cur]['Изменение за 24 часа'].pop(0)
                chart[base_cur]['Изменение за 24 часа'].append(round(value, 2))

    if previous_crypto_json:
        for base_cur in crypto.values():
            if base_cur not in crypto_json and base_cur not in previous_crypto_json:
                continue
            else:
                rate_prev = previous_crypto_json[base_cur]['usd']
                rate = crypto_json[base_cur]['usd']
                change = (rate - rate_prev) / rate_prev * 100

                values_yaxis.append(int(change))  # добавление значения в список для мин/макс
оси Y

                if len(chart[base_cur]['Изменение с предыдущего запроса']) < 10:
                    chart[base_cur]['Изменение с предыдущего запроса'].append(round(change, 2))
                else:
                    chart[base_cur]['Изменение с предыдущего запроса'].pop(0)
                    chart[base_cur]['Изменение с предыдущего запроса'].append(round(change, 2))

    if values_yaxis:  # сдвиг значений МИН/МАКС, используемых для графика по оси X
        if len(values_yaxis) > 240:
            rest_of_len = len(values_yaxis) % 240
            values_yaxis = values_yaxis[rest_of_len:]
        max_value_yaxis = max(values_yaxis)
        min_value_yaxis = min(values_yaxis)


def diagram():
    ax.clear()  # очистка графика

    base_c = crypto_combo.get()
    item = crypto[base_c]  # название криптовалюты из словаря валют - 'bitcoin'

    if item in chart:
        change24_items = chart[item]['Изменение за 24 часа']
        change_last_items = chart[item]['Изменение с предыдущего запроса']

        # графики
        ax.plot(change24_items, marker='o', label='за 24 часа')
        ax.plot(change_last_items, marker='o', label='с предыдущего запроса')

        if max(change24_items) >= max(change_last_items): # расположение меток значений
```

```
относительно графика
            xytext_change24 = (0, 10)
            xytext_change_last = (0, -15)
        else:
            xytext_change24 = (0, -15)
            xytext_change_last = (0, 10)

        for i, value in enumerate(change24_items):  # подписи значений
            ax.annotate(f'{value}%', (i, value),
                        textcoords="offset points", xytext=xytext_change24, ha='center',
                        fontsize=8, color='DodgerBlue')

        for i, value in enumerate(change_last_items):  # подписи значений
            ax.annotate(f'{value}%', (i, value),
                        textcoords="offset points", xytext=xytext_change_last, ha='center',
                        fontsize=8, color='OrangeRed')

        ax.set_title(f'Изменение курса {base_c}")
        ax.set_xlabel("Время обновления")
        ax.set_ylabel("Изменение, %")

        ax.set_xlim(-0.3, 9.3)
        ax.set_xticks(range(10))

        ax.set_ylim(int(min_value_yaxis) - 2, int(max_value_yaxis) + 2)
        ax.tick_params(axis='both', labelsize=8, labelcolor='gray')  # изменение шрифтов осей
        ax.set_xticklabels(renew_time)  # названия по оси X
        ax.legend()

    ax.grid(True)
    fig.tight_layout()  # Matplotlib автоматически настраивает отступы между подграфиками
    # (субплотами) и вокруг них, чтобы элементы — подписи осей, заголовки,
    # метки тиков — не накладывались друг на друга
    canvas.draw_idle()  # перерисовка графика


def start_refresh():
    """Старт обновления курсов с сайта и запуск таймера обратного отсчета на
кнопке"""
    global time_to_refresh, countdown

    countdown = None

    target_field.config(text='Загрузка курсов с сайта coingecko.com... ')
    refresh_btn.config(text='Обновление...')
    refresh_label.config(text='Загрузка... ')
    refresh_btn.config(state='disabled')

    root.update()
    sleep(2)
    refresh_rates()
```

```
    refresh_btn.config(state='normal')

    time_to_refresh = random.randint(61, 71)
    countdown = True
    button_countdown()


def button_countdown():
    """Таймер обратного отсчета на кнопке обновления курсов валют"""
    global time_to_refresh

    toggle_flag = flag.get()

    if countdown is None:
        return

    if toggle_flag:
        time_to_refresh -= 1
        if time_to_refresh > 0:
            refresh_btn.config(text=f'Автообновление курсов через {time_to_refresh} сек.')
            root.after(1000, button_countdown)
        else:
            refresh_btn.config(text='Обновление...')
            root.after(time_to_refresh * 1000, start_refresh)
    else:
        refresh_btn.config(text='Обновить курсы для списка криптовалют')


def refresh_rates():
    """Получение курсов с сайта в формате json"""
    global crypto_json, previous_crypto_json, renew_time

    previous_crypto_json = copy.deepcopy(crypto_json)

    url = "https://api.coingecko.com/api/v3/simple/price"
    params = {
        "ids": ",".join(crypto.values()),
        "vs_currencies": "usd",
        "include_24hr_change": "true",
    }
    try:
        response = requests.get(url, params=params, timeout=10)
        response.raise_for_status()
        crypto_json = response.json()

        now = datetime.now()
        day = now.strftime("%d.%m.%Y")
        hours = now.strftime("%H:%M:%S")
        hours_hm = now.strftime("%H:%M")
        refresh_label.config(text=f'Данные обновлены:  {day}  {hours}')

        if '' in renew_time:  # обновление списка времени загрузки данных для подписей оси X
```

```
на графике
            old = renew_time.index('')
            renew_time[old] = hours_hm
        else:
            renew_time = renew_time[1:] + [hours_hm]

    except Exception as e:
        mb.showerror('Ошибка', f'Ошибка загрузки курсов: {e}')
        target_field.config(text='Ошибка при загрузке курсов', font=('Verdana', 11))
        refresh_label.config(text='Данные обновлены:   --.--.----   --:--:--')

    add_to_chart()
    target_field_update()


crypto = {
    'Bitcoin (BTC)': 'bitcoin',
    'Binancecoin (BNB)': 'binancecoin',
    'Bitcoin-cash (BCH)': 'bitcoin-cash',
    'Bittensor (TAO)': 'bittensor',
    'Ethereum (ETH)': 'ethereum',
    'Hyperliquid (HYPE)': 'hyperliquid',
    'Monero (XMR)': 'monero',
    'Pax-gold (PAXG)': 'pax-gold',
    'Quant-network (QNT)': 'quant-network',
    'Solana (SOL)': 'solana',
    'Tether-gold (XAUT)': 'tether-gold',
    'Zcash (ZEC)': 'zcash'
}

chart = {}  # словарь 10 последних запросов: изменение курса за 24 часа и с предыдущего
запроса
crypto_json = {}  # json-файл полученный от сервера
previous_crypto_json = {}  # копия предыдущего json-файла для сравнения курсов
time_to_refresh = 0
countdown = None
renew_time = ['', '', '', '', '', '', '', '', '', '']
values_yaxis = []  # список значений изменений котировок для поиска мин/макс
max_value_yaxis = 0  # значение изменений котировок для установки МАКС значения оси
Y
min_value_yaxis = 0  # значение изменений котировок для установки МИН значения оси Y

root = Tk()
root.title('Курсы популярных криптовалют')
w = root.winfo_screenwidth()
h = root.winfo_screenheight()
w2 = w // 6
h2 = h // 4
root.geometry(f'1000x430+{w2}+{h2}')

# Left frame ----------------------------------------
left = Frame(root, width=410)
```

```
left.pack(side="left", fill="y")
left.pack_propagate(False)  # не сжиматься под содержимое

# frame1 ------------------------------------------
frame1 = Frame(left, bg='lightgray')
frame1.pack(padx=10, pady=4, fill='both')

base_l = Label(frame1, text='Список криптовалют', font=('Verdana', 10),
           bg='lightgray')  # Заголовок выпадающего списка
base_l.pack(pady=(10, 5))
crypto_combo = ttk.Combobox(frame1, values=list(crypto.keys()), state='readonly')  #
Выпадающий список криптовалют
crypto_combo.current(0)
crypto_combo.pack(padx=10, pady=(5, 10), fill='both')
crypto_combo.bind('<<ComboboxSelected>>', target_field_update)

# frame2 ------------------------------------------
frame2 = Frame(left, bg='lightgray')
frame2.pack(padx=10, pady=4, fill='both')

target_l = Label(frame2, text='Целевая валюта', bg='lightgray', font=('Verdana', 10))
target_l.pack(padx=10, pady=(10, 5))

target_USD = Label(frame2, text='USD', bg='lightgray', font=('Verdana', 10))
target_USD.pack(padx=10, pady=(5, 10))

# frame3 ------------------------------------------
frame3 = Frame(left, bg='lightgray')
frame3.pack(padx=10, pady=(4, 0), fill='both')

target_field = Label(frame3, text='Загрузка курсов с сайта coingecko.com... ', font=('Verdana',
11, 'bold'), height=3,
                bg='Lavender')  # Поле для вывода текущего курса
target_field.pack(padx=10, pady=10, fill='both')

# frame4 ------------------------------------------
frame4 = Frame(left, bg='lightgray')
frame4.pack(padx=10, pady=(0, 4), fill='both')

change_label = Label(frame4, text='Изменение курса за последние 24 часа', bg='lightgray',
                font=('Verdana', 10))
change_label.grid(row=0, column=0, padx=(10, 0), pady=(5, 5), sticky='w')

change24_value = Label(frame4, text='-', bg='lightgray', font=('Verdana', 10, 'bold'), width=8)
change24_value.grid(row=0, column=2, sticky='e')

last_change_label = Label(frame4, text='Изменение курса с предыдущего запроса',
bg='lightgray',
                    font=('Verdana', 10))
last_change_label.grid(row=1, column=0, padx=(10, 0), pady=(5, 10), sticky='w')

last_change_value = Label(frame4, text='-', bg='lightgray', font=('Verdana', 10, 'bold'), width=8)
```

```
last_change_value.grid(row=1, column=2, sticky='e')

# frame5 ----------------------------------------
frame5 = Frame(left, bg='lightgray')
frame5.pack(padx=10, pady=(4, 0), fill='both')

refresh_btn = Button(frame5, text='Обновление...',
                     command=start_refresh, bg='Lavender', font=('Verdana', 10))
refresh_btn.pack(padx=10, pady=(10, 0), fill='both')

flag = BooleanVar(value=True)
auto_scenario = Checkbutton(frame5, text="Включить / отключить автообновление",
                            bg='lightgray', variable=flag, offvalue=False)
auto_scenario.pack(pady=3)

refresh_label = Label(frame5, text='Загрузка... ', fg='DimGray')
refresh_label.pack(pady=(4, 0), fill='both')

# frame6 right---------------------------------------
frame6 = Frame(root, bg='lightgray')
frame6.pack(side="left", padx=(0, 10), pady=(4, 10), fill='both', expand=True)

fig = Figure(figsize=(10, 5), dpi=100)
ax = fig.add_subplot(111)
canvas = FigureCanvasTkAgg(fig, master=frame6)  # встраиваем график в Tkinter
canvas.get_tk_widget().pack(fill=BOTH, expand=True)

start_refresh()

root.mainloop()
```

7. ЗАКЛЮЧЕНИЕ

Разработанное программное обеспечение «Курсы популярных криптовалют» позволит упростить пользователям поиск информации о популярных валютах и имеет коммерческий потенциал. В перспективе программа может быть доработана для вывода курса нескольких целевых валют.